

# TN+NTN Hybrid Network Project - Consolidated Report

Mohamed Hassan; Omar Shawky; Mohamed Essam

2026-08-02

## Contents

|                                                                              |          |
|------------------------------------------------------------------------------|----------|
| <b>TN+NTN Hybrid Network Project — Consolidated Report</b>                   | <b>1</b> |
| 1. What this project is . . . . .                                            | 1        |
| 2. Repository layout . . . . .                                               | 2        |
| 3. Methodology . . . . .                                                     | 2        |
| 4. The plan, and where we are . . . . .                                      | 4        |
| 5. Phase-by-phase record . . . . .                                           | 5        |
| 6. Results . . . . .                                                         | 6        |
| 7. Known issues, gaps, and limitations . . . . .                             | 8        |
| 8. Parallel direction — Federated Learning (assessed, not started) . . . . . | 8        |
| 9. Artifact index . . . . .                                                  | 9        |
| 10. Reproduction . . . . .                                                   | 11       |
| 11. Housekeeping record (2026-08-02) . . . . .                               | 12       |

## TN+NTN Hybrid Network Project — Consolidated Report

**Authors:** Mohamed Hassan, Omar Shawky, Mohamed Essam **Last updated:** 2026-08-02 **Scope:** Everything built to date, the methodology behind it, the plan, and current position in that plan.

This is the single source of truth for the project. It supersedes the former long-form README.md, status.md, and PROJECT\_REFERENCE.md, which have been merged here.

---

### 1. What this project is

A simulation-and-learning stack for **hybrid Terrestrial (TN) / Non-Terrestrial (NTN) networks** covering five radio access technologies: 5G NR, WiFi 6, HAPS, LEO satellite, and UAV relay.

It answers one core research question:

Given a mobile user that can reach several of these networks at once, which network should it connect to at each moment — maximizing quality of experience while avoiding wasteful handovers?

The stack has two halves:

- **A physics-grounded simulator** that produces realistic per-link radio conditions.
  - **A deep reinforcement learning agent** that learns a network-selection policy on top of it, benchmarked against classical telecom handover algorithms.
- 

## 2. Repository layout

|                                    |                                                                                                                       |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| TN+NTN/                            |                                                                                                                       |
| └─ REPORT.md                       | this file (the project record)                                                                                        |
| └─ README.md                       | short orientation + quick start                                                                                       |
| └─ data/                           |                                                                                                                       |
| └─ Hybrid_Network_TN_NTN_Final.csv | merged 5-network dataset                                                                                              |
| └─ raw/                            | clean per-technology scenario CSVs                                                                                    |
| └─ regenerated/                    | fresh re-run of layer 1, for comparison                                                                               |
| └─ sim/                            |                                                                                                                       |
| └─ python_sim_all.py               | layer 1 - scenario simulator                                                                                          |
| └─ ns3/                            | layer 2 - NS-3 scripts (planned)                                                                                      |
| └─ noise_models/                   |                                                                                                                       |
| └─ channels/                       | layer 3 - physics/ITU-R channel model<br>thermal, pathloss, fading, doppler,<br>atmosphere, interference, impairments |
| └─ drl/                            |                                                                                                                       |
| └─ figures/                        | reinforcement learning package<br>result figures, CSVs, metric summaries                                              |
| └─ *.d3 / *.zip                    | trained Phase-1/Phase-2 models                                                                                        |
| └─ scripts/                        | runnable drivers                                                                                                      |
| └─ validation/                     | invariant checks + diagnostic plots                                                                                   |
| └─ output/                         | noisy datasets (parquet gitignored)                                                                                   |
| └─ d3rlpy_logs/                    | raw CQL per-step training logs                                                                                        |
| └─ docs/                           | noise_model_research.md                                                                                               |
| └─ notebooks/                      | original EDA/merge notebook                                                                                           |
| └─ reports/                        | formal write-ups + this report as PDF                                                                                 |

Package internals: noise\_models/ holds config.py, constants.py, technologies.py, link.py, generate.py, validate.py; drl/ holds features.py, reward.py, mobility.py, env.py, baselines.py, evaluation.py.

---

## 3. Methodology

### 3.1 Three simulation layers

The simulator goes from coarse geometry down to fine-grained channel realism.

**Layer 1 — system-level scenario simulator** (sim/python\_sim\_all.py) A lightweight Python simulator modelling UE mobility, beam/cell association, and simplified per-technology channel formulas. Produces baseline scenario datasets (distance, Doppler, rain rate, SNR/SINR, ...) at coarse resolution: 10-second steps over a ~6,000-second trace, 20 UEs per technology. Output lives in data/raw/.

**Layer 2 — NS-3 protocol-level simulation** (sim/ns3/) — *planned, not yet integrated* Packet- and protocol-level simulation (scheduling, retransmission, handover) layered on the same scenarios. Four exact-parameter .cc scripts exist; they are not yet wired into the pipeline.

**Layer 3 — physics/standards-grounded noise model** (noise\_models/) The core of the simulator. Each coarse scenario row is treated as a **geometry descriptor** and regenerated into a realistic **6-second, 10 ms-resolution time series**, propagating impairments coherently through the full link chain:

thermal noise → path loss → shadowing → Rayleigh/Rician fading → Doppler  
→ ITU-R rain / gaseous / scintillation attenuation → interference → hardware limits  
→ SNR → SINR → BER → throughput → packet loss → latency → link-quality index

Standards referenced, per-technology RF assumptions, chain equations, calibration results, and every documented simplification are recorded in docs/noise\_model\_research.md. Physical invariant checks and diagnostic plots (CDF overlays, fading autocorrelation vs. theory, rain curves, time traces) are in validation/.

## 3.2 The learning problem

**State design — the decisive early finding.** The physical-layer KPI columns (distance, SNR, SINR, throughput, latency) are *entirely determined by which network a row represents*, not by shared pre-decision context. Confirmed empirically: distance\_km is ~800-1900 km for every SAT (LEO) row versus under ~2 km for every NR\_5G row. Feeding these as state would leak the answer.

The discipline adopted throughout, and never broken: **only pre-decision measurables enter the observation; outcomes stay reward-only.**

**Feature approach — Option A.** Engineered physical-layer KPIs are fed directly as state; the model is *not* asked to reconstruct KPIs from raw physics.

**Reward — composite QoE** (drl/reward.py). A weighted combination over throughput, latency, and reliability, with normalization bounds fitted at the 1st-99th percentile per KPI and persisted to drl/reward\_norm\_params.json for reproducibility. Latency\_ms is used directly because it already includes propagation delay (noise\_models/link.py:135) — which is precisely what separates fast TN from slow NTN links. In the sequential setting a flat **handover penalty** (default 0.15) is subtracted whenever the agent changes network.

**Environment** (drl/env.py, NetworkSelectionEnv, Gymnasium). Observation is 26-dimensional: Area one-hot + previous-action one-hot + per-candidate-network [available, RSSI\_norm, SINR\_norm], all computed live each step via

`noise_models/link.py:simulate_trace`. Action is the 5-way network choice. Selecting an unavailable network is invalid (-1.0 reward, no state advance). The env passes `gymnasium.utils.env_checker.check_env`.

**Shared-mobility model** (`drl/mobility.py`). Needed because the source data has no shared UE trajectory across networks (see §5, Phase 2). It provides:

- Bounded random-walk geometry for TN/UAV candidates, within 1st-99th-percentile ranges fitted per `network_type` from the merged CSV, so all candidates are evaluated against the *same* simulated moment.
- Realistic **LEO/HAPS pass geometry** — elevation rises, peaks, and sets, with `distance_km` derived from elevation via the inverse of `noise_models/channels/pathloss_ele`. Below `MIN_VISIBLE_ELEV_DEG = 10°` the platform is out of view, forcing a genuine handover. Measured: LEO toggles visibility ~12 times per 200 steps; HAPS is calibrated as a persistent regional platform, visible ~79% of the time.
- **Area evolves within an episode** via a hand-specified Markov chain (`AREA_TRANSITION`).

### 3.3 Evaluation methodology

`drl/evaluation.py` provides `rollout_episode`, `evaluate_policy`, `bootstrap_ci` (2,000 resamples), `summarize_eval`, and `multi_seed_evaluate`. Any policy exposing `.predict(obs) -> int` can be evaluated; SB3Policy and D3RLPolicy adapters are included.

Protocol: **3 training seeds per algorithm × 200 evaluation episodes per seed = 600 episodes**, with bootstrap 95% confidence intervals. Reported metrics are mean reward, switch (handover) rate, and outage (invalid-action) rate.

**Baselines** (`drl/baselines.py`) are deliberately *telecom handover algorithms*, not an offline-RL zoo — the comparison that matters for this domain:

- RandomPolicy — floor.
- GreedyPolicy — myopic best-instant choice; a ping-pong strawman.
- HysteresisPolicy — 3GPP A3-style margin + time-to-trigger; the serious classical competitor.
- oracle — dynamic programming over a reward matrix built with independent-fading planning noise, then scored through the env’s real per-seed-deterministic stepping. (See the docstring in `drl/baselines.py` for why the separate planning-noise draw still makes this a fair upper bound.)

## 4. The plan, and where we are

| Phase | Goal                                                         | Status |
|-------|--------------------------------------------------------------|--------|
| 1     | Contextual-bandit framing; offline CQL on the merged dataset | Done   |

| Phase | Goal                                                           | Status                   |
|-------|----------------------------------------------------------------|--------------------------|
| 2     | Sequential DRL with handover cost; Gym env + PPO               | Done (superseded by 3-5) |
| 3     | Condition-aware environment (live radio measurements in state) | Done                     |
| 4     | Rigorous evaluation harness (multi-seed, bootstrap CIs)        | Done                     |
| 5     | Telecom baselines + RL comparison                              | Done                     |
| 6     | Reward / handover-cost sensitivity sweep                       | Done                     |
| —     | <b>Phases 1-6 plan complete</b>                                | <b>Complete</b>          |
| 7     | NS-3 protocol-level integration (layer 2)                      | Not started              |
| 8     | Federated learning direction (parallel track)                  | Assessed, not started    |

**Current position: the Phases 1-6 plan is complete.** Condition-aware RL agents have been benchmarked against classical telecom handover baselines with multi-seed 95% CIs (Phase 5), plus a documented reward/handover-cost sensitivity analysis (Phase 6). The two open directions are NS-3 integration and the federated-learning track (§8).

## 5. Phase-by-phase record

### Phase 1 — contextual bandit (done)

State = Area + Available\_Networks only, on the grounds that KPI columns are outcomes of the network choice rather than pre-decision context. Action = network\_type (5-way). Reward = composite QoE. Trained **Discrete CQL** (d3rlpy) on data/Hybrid\_Network\_TN\_NTN\_Final.csv.

Finding:  $\alpha=0.1$  (down from the default 1.0) matches the empirical-best network in **5 of 6 Areas**; the default alpha matched only 3 of 6 — it over-penalized on this small, fully-covered 6-state action table. Model: drl/cql\_network\_selector\_alpha0.1.d3.

Reward sanity: range [0.0066, 0.9599], mean 0.6535 over 55,503 rows.

### Phase 2 — sequential DRL, first attempt (done, later superseded)

**Gap identified:** noise\_models/generate.py produces only one technology's trace per source row, drawn from an *independent* Monte-Carlo scenario per technology

(confirmed in `sim/python_sim_all.py` — `sim_5g/sim_haps/sim_leo/etc.` each simulate their own unrelated 20 UEs). There was **no shared UE trajectory across networks**, so handover comparison had no physical basis. Resolved by building `drl/mobility.py` (§3.2) and `drl/env.py`, then training PPO.

### Phases 3-5 — paper-grade redesign (done)

Re-evaluating Phase 2 against the “academic paper” goal surfaced a **foundational flaw**: its observation (`Area + Available_Networks + prev-action`) carried no live radio measurements, so the policy could only ever be a static per-Area lookup plus stickiness — blind to actual conditions, undermining the case for deep RL at all. Fixed by a full redesign (confirmed with the user: enrich the state; use telecom baselines, not an offline-RL zoo):

- **Phase 3** — condition-aware env and realistic pass geometry (§3.2).
- **Phase 4** — evaluation harness with bootstrap CIs (§3.3).
- **Phase 5** — telecom baselines + PPO and DQN (`stable_baselines3`), 3 seeds each, 100k steps per seed, 200 evaluation episodes per seed.

### Phase 6 — sensitivity sweep (done)

`scripts/sweep_reward_handover.py` runs **two 1-D sweeps** (not a full grid, to bound retraining cost), retraining a fresh PPO at each point (50k steps, reduced from Phase 5’s 100k to keep the 8-point sweep tractable):

- Handover penalty  $\in \{0.05, 0.15, 0.30, 0.50\}$  at balanced reward weights.
- Reward-weight profile  $\in \{\text{balanced, throughput-leaning, latency-leaning, reliability-leaning}\}$  at the default penalty (0.15).

## 6. Results

### 6.1 Headline comparison (Phase 5)

Mean reward [95% CI] / switch rate / outage rate. Source: `drl/figures/phase5_summary.csv`.

| Policy                     | Reward                      | Switch rate  | Outage rate |
|----------------------------|-----------------------------|--------------|-------------|
| random                     | 3.17 [2.08, 4.22]           | 0.413        | 0.332       |
| greedy                     | 39.88 [39.60, 40.17]        | 0.170        | 0.000       |
| hysteresis (3GPP A3-style) | 40.88 [40.59, 41.19]        | 0.062        | 0.000       |
| dqn                        | 43.12 [42.92, 43.33]        | 0.159        | 0.000       |
| <b>ppo</b>                 | <b>44.19 [43.97, 44.42]</b> | <b>0.043</b> | 0.0001      |
| oracle (upper bound)       | 45.20 [44.83, 45.56]        | 0.051        | 0.000       |

**Headline finding.** PPO reaches ~98% of the oracle’s reward (regret  $\approx 1.01$ , ~2%) while achieving the **lowest switch rate of any policy — including the hand-tuned hysteresis baseline**. It learned implicit anti-ping-pong behaviour purely from the

handover-cost reward term, beating an explicit rule-based algorithm on *both* reward and switch rate.

**An honest asymmetry worth reporting:** DQN clearly beats the classical baselines on reward but *not* on switch rate (0.159, close to greedy’s 0.170). The two RL algorithms differ meaningfully here, and this is reported rather than smoothed over.

## 6.2 Handover-penalty sensitivity (Phase 6)

Source: `drl/figures/phase6_handover_sweep.csv` / `.png` (balanced reward weights).

| Handover penalty | Reward | Switch rate |
|------------------|--------|-------------|
| 0.05             | 44.31  | 0.082       |
| 0.15             | 43.83  | 0.057       |
| 0.30             | 43.38  | 0.047       |
| 0.50             | 43.15  | 0.030       |

Switch rate is **monotonically non-increasing** as the penalty rises (verified True by the script’s own check) — the mechanism works as designed. The small reward cost as the penalty tightens (44.31 → 43.15) is a legitimate trade-off to report, not a bug.

## 6.3 Reward-profile sensitivity (Phase 6)

Source: `drl/figures/phase6_reward_profile_sweep.csv` / `.png` (handover penalty = 0.15).

| Reward profile         | Reward       | Switch rate |
|------------------------|--------------|-------------|
| balanced               | 43.83        | 0.0565      |
| throughput-leaning     | 43.48        | 0.0528      |
| <b>latency-leaning</b> | <b>44.86</b> | 0.0507      |
| reliability-leaning    | 43.18        | 0.0465      |

Latency-leaning is clearly best on reward — consistent with the reward-design finding that `Latency_ms` (which already includes propagation delay) is the single KPI that most sharply separates TN from NTN networks, giving the agent the clearest signal to act on.

**Consistency check passed:** the balanced row here (43.83, 0.0565) exactly matches the `handover_penalty=0.15` row in the other sweep — the same configuration evaluated twice, identical result.

## 6.4 Key figures

`drl/figures/phase5_reward_vs_switch_tradeoff.png` is the cleanest single figure for a paper. Also: `phase5_reward_comparison.png`, `phase5_switch_outage_comparison.png`, `phase6_handover_sweep.png`, `phase6_reward_profile_sweep.png`.

---

## 7. Known issues, gaps, and limitations

**Inherited defects** (documented in reports/TN\_NTN\_methodology\_bugs.pdf):

- notebooks/TN\_NTN\_Deep\_RL\_MODEL.ipynb contains **no RL code** — EDA and merging only, despite its title.
- No feature extraction/selection was ever implemented; reports/feature\_TN\_NTN.pdf is a design spec only.
- The merged CSV has **nonzero rain features for NR\_5G** despite the spec marking rain as not applicable to 5G. Still unresolved; low priority.

**Modelling simplifications (deliberate, documented):**

- Doppler is held at each technology's median in the mobility model — no orbital or positional mechanics are simulated.
- All five source datasets come from one simulation pipeline, so the data is single-source synthetic.

**Instrumentation gaps:**

- Phase 5 PPO/DQN **model weights are not saved to disk** — scripts/train\_agents.py trains 3 seeds in memory and persists only the *results* (phase5\_full\_eval\_results.csv). Rerun to reproduce.
- **SB3 (PPO/DQN) training curves are not persisted.** Unlike CQL (which writes per-step CSVs to d3rlpy\_logs/), SB3's live progress tables exist only in whatever console log was being watched. Fix is one line — pass `tensorboard_log=` to `PP0(...)/DQN(...)`, or wrap envs in SB3's `Monitor(filename=...)`. This blocks cross-algorithm sample-efficiency plots (PPO vs. DQN convergence speed), though not the headline comparison. Flagged as a known gap, not a silent omission.
- PPO's `explained_variance` stayed low (~0.01-0.15) across runs — a real limitation of the value function worth reporting alongside the results.
- Not computed: wall-clock/compute cost per algorithm; action distribution / policy entropy at convergence; inter-seed standard deviation reported separately from within-policy episode variance (the raw per-seed data *is* in `phase5_full_eval_results.csv:train_seed`).

**Bottom line on gaps:** everything needed for the headline comparison (§6.1) is solid and reproducible. The one real gap is unpersisted PPO/DQN training curves.

---

## 8. Parallel direction — Federated Learning (assessed, not started)

Full analysis in reports/FL\_TN\_NTN\_Assessment.md / .pdf. Summary:

**Why the data suits an FL study.** Organic (not synthetically injected) non-IID data — the tiers differ *physically* (LEO's large Doppler swings vs. near-zero terrestrial; WiFi's sub-metre distances vs. LEO's 800+ km). Real measured `Propagation_Delay_ms` per

tier enables genuine communication-cost comparisons, which most FL-in-NTN work only assumes. Clean vHetNet three-tier structure (terrestrial / aerial / space) gives a defensible client partition.

**The hard constraint.** UE identifiers do **not** align across files: four of five files reuse UE\_000-UE\_019 and WiFi has 7,583 distinct ones, but matching labels are **not the same physical user** — each tier came from an independent Monte-Carlo run. A “one UE across all tiers” client design is therefore **not scientifically valid**; the only honest partition is  $client = tier/RAT$ , which forces a five-client federation. At that scale client-selection methods are meaningless and FedAvg-vs-FedProx differences are marginal, so flat FL is weak.

**Recommended direction.** Decentralized (peer-to-peer gossip) FL, which both repairs the five-client limitation (multiple nodes per tier → tens of nodes) and targets what DFL most lacks: a **realistic intermittent, partitioning, latency-heterogeneous topology**. This project’s simulator — `noise_models/` plus the pass-geometry and visibility model in `drl/mobility.py` — is precisely the asset that makes such a topology credible, where most DFL work hand-waves a synthetic random graph.

**Blocking prerequisite:** a label/utility function (“which RAT is optimal for a given KPI row”) must be defined as a concrete formula before *any* FL training work, centralized baseline included.

---

## 9. Artifact index

### Trained models

| Path                                              | What it is                                                                                           |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>drl/cql_network_selector_alpha0.1.d3</code> | Phase 1 CQL, tuned $\alpha=0.1$ (5/6 Area match) — load via <code>d3rlpy.load_learnable(path)</code> |
| <code>drl/cql_network_selector.d3</code>          | Phase 1 CQL, default $\alpha=1.0$ (superseded, 3/6 match)                                            |
| <code>drl/ppo_network_selector.zip</code>         | Phase 2 thin-state PPO (superseded; kept for history)                                                |
| Phase 5 PPO/DQN                                   | <b>Not persisted</b> — see §7                                                                        |

### Source package (drl/)

| File                     | Contents                                                                 |
|--------------------------|--------------------------------------------------------------------------|
| <code>features.py</code> | NETWORK_TYPES, AREAS;<br>build_state/build_actions;<br>parse_observation |
| <code>reward.py</code>   | RewardWeights, fit_normalization,<br>compute_reward                      |

| File          | Contents                                                                                                                                            |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| mobility.py   | Bounded-walk geometry, LEO/HAPS pass geometry, AREA_TRANSITION                                                                                      |
| env.py        | NetworkSelectionEnv — observation, handover penalty, invalid-action handling                                                                        |
| baselines.py  | RandomPolicy, GreedyPolicy, HysteresisPolicy,                                                                                                       |
| evaluation.py | make_oracle_policy + DP solver<br>rollout_episode, evaluate_policy,<br>bootstrap_ci, summarize_eval,<br>multi_seed_evaluate, SB3/d3rlpy<br>adapters |

### Runnable drivers (scripts/)

| Script                   | Produces                                                       |
|--------------------------|----------------------------------------------------------------|
| run_all.py sample\ full  | Layer-3 noisy datasets → output/, plots + report → validation/ |
| compute_reward.py        | drl/reward_norm_params.json + reward sanity summary            |
| train_cql.py             | Phase 1 CQL model + d3rlpy_logs/DiscreteCQL_*/                 |
| train_ppo.py             | Phase 2 PPO (superseded, thin state)                           |
| make_drl_figures.py      | Phase 1-2 figures + drl/figures/METRICS.md                     |
| train_agents.py          | Phase 5 baselines + multi-seed PPO/DQN → phase5_*.csv          |
| make_phase5_figures.py   | Phase 5 comparison figures                                     |
| sweep_reward_handover.py | Phase 6 sweeps → phase6_*.csv / .png                           |

### Results data (drl/figures/)

phase5\_full\_eval\_results.csv is the **source-of-truth raw table** (all 6 policies, 200 episodes × up to 3 seeds each). phase5\_summary.csv is the aggregated mean + 95% CI table reproduced in §6.1. phase6\_handover\_sweep.csv and phase6\_reward\_profile\_sweep.csv back §6.2 and §6.3. METRICS.md holds plain-text Phase 1-2 headline numbers.

### Formal write-ups (reports/)

| File                      | Contents                                                                 |
|---------------------------|--------------------------------------------------------------------------|
| TN_NTN_Project_Report.pdf | PDF rendering of this report (regenerate with the pandoc command in §10) |

| File                                         | Contents                                                  |
|----------------------------------------------|-----------------------------------------------------------|
| TN_NTN_DRL_Report.md / .pdf                  | The main paper: DRL methodology and evaluation            |
| FL_TN_NTN_Assessment.md / .pdf               | Federated-learning assessment and decentralized direction |
| TN_NTN_methodology_bugs.pdf                  | Bug report on the inherited notebook and feature spec     |
| feature_TN_NTN.pdf                           | Original feature design spec (never implemented)          |
| Package_C_Federated_TN-NTN_Deep_Dive.md.docx | Federated deep-dive source document                       |

## 10. Reproduction

```
pip install numpy pandas scipy matplotlib pyarrow d3rlpy stable-baselines3
```

```
# Layer 1 – regenerate baseline scenario datasets
```

```
python sim/python_sim_all.py
```

```
# Layer 3 – regenerate fine-grained noisy time series
```

```
python scripts/run_all.py sample # fast: samples + validation/report.md
```

```
python scripts/run_all.py full # complete generation, ~33M rows
```

```
# DRL pipeline
```

```
python scripts/compute_reward.py # fit + persist reward normalization
```

```
python scripts/train_cql.py # Phase 1
```

```
python scripts/train_agents.py # Phase 5 (long: 3 seeds x 2 algorithms)
```

```
python scripts/make_phase5_figures.py
```

```
python scripts/sweep_reward_handover.py # Phase 6 (long: 8 retrainings)
```

## Rebuilding this report as a PDF

```
pandoc REPORT.md -o reports/TN_NTN_Project_Report.pdf \
  --pdf-engine=xelatex --toc --toc-depth=2 \
  -V geometry:margin=1in -V fontsize=11pt \
  -V colorlinks=true -V linkcolor=blue -V urlcolor=blue \
  -V mainfont="DejaVu Sans" -V monofont="DejaVu Sans Mono" \
  -V "header-includes=\emergencystretch=3em" \
  --metadata title="TN+NTN Hybrid Network Project - Consolidated Report" \
  --metadata author="Mohamed Hassan; Omar Shawky; Mohamed Essam" \
  --metadata date="2026-08-02"
```

Keep the report free of emoji and other glyphs outside the DejaVu fonts, or the xelatex pass will drop them silently.

## Environment notes

- **Windows has two Python installs on PATH:** pip resolves to Python 3.13's, but python resolves to 3.14 (AppData\Local\Python\pythoncore-3.14-64). Always install with `python -m pip install`, never bare `pip install`, or packages land in the wrong interpreter.
  - **Background training runs buffer stdout.** Once redirected to a file, `print()` is block-buffered, so output can sit invisible for many minutes while the process works. Launch with `python -u` **and** add explicit progress prints inside long loops — `-u` alone is not enough if the loop never prints until it finishes. `drl/evaluation.py:evaluate_policy` (via `progress_label/progress_every`) and the baseline/oracle loops in `scripts/train_agents.py` already do this.
- 

## 11. Housekeeping record (2026-08-02)

The repository was reorganized on this date:

- Data consolidated under `data/` (`raw/`, `regenerated/`, and the merged CSV).
- Five root-level CSVs that were **byte-identical duplicates** of `Dataset/` were deleted after checksum verification; `Dataset/` became `data/raw/`.
- Simulator code consolidated under `sim/` (`Script python/` → `sim/`, `Scripts on Ns3/` → `sim/ns3/`), removing spaces from directory names.
- Papers moved to `reports/`, the notebook to `notebooks/`.
- Hardcoded data paths updated in 7 scripts (`run_all.py`, 2 occurrences, plus `CSV_PATH` in `compute_reward`, `make_drl_figures`, `sweep_reward_handover`, `train_agents`, `train_cql`, `train_ppo`).
- The long-form `README.md`, `status.md`, and `PROJECT_REFERENCE.md` were merged into this report.
- This report is also rendered to `reports/TN_NTN_Project_Report.pdf` (see §10).

Verified after the move: `scripts/compute_reward.py` reproduces its recorded output exactly (range `[0.0066, 0.9599]`, mean `0.6535`, 55,503 rows) and `drl/reward_norm_params.json` is unchanged; `noise_models.generate_noisy_dataframe` smoke-tests clean against `data/raw/`.